Proposal: Fireteam: Skirmish

Course: CSC2210 Procedural and Object-Oriented C++ **Team:** Aditya Miriyala, Khushal Lingamaneni, Koushik Cherukuri

1. Overview

Fireteam: Skirmish is a real-time, team-based tactical shooter for small LAN groups. It is for classmates and friends who want a fast, casual competitive shooter they can host and play together over a local network/Wi-Fi, without a dedicated server, matchmaking service, or commercial game client. The problem it solves: there is no lightweight way for a small group of friends to drop into a quick team shootout using just their own laptops on the same network.

2. Core features (must work)

1. **Real-time networked gameplay** (2+ human players over LAN via ENet) — position, aim, and shooting state replicated between clients.
2. **Player entity**: movement, aiming, a basic hitscan weapon.
3. **Hit detection & damage**: shots reduce opponent health consistently on all clients; health reaching 0 eliminates the player.
4. **Win condition**: match ends and declares a result when one side/player remains.
5. **Clean OOP structure**: `Entity` base class with polymorphic `Player`, an RAII-wrapped `NetworkManager`, and a `GameState` state machine (Lobby → InGame → MatchOver).

Verification: run two client instances on the same LAN; confirm synced positions within ~100ms; confirm a shot reduces health identically on both clients; confirm eliminating all opposing players ends the match and shows a result screen.

Stretch goals (not required for core grade)

- AI-controlled bots filling empty slots (simple sight-based engage/patrol logic), to scale toward 5v5
- Grenade/explosive weapon with real projectile physics (gravity, bounce, blast radius)
- Full 5v5 scale, HUD polish, sound, multiple maps

3. Design sketch

- `Entity` (abstract base: position, health, virtual `update()` / `render()`) → `Player` (derived), and stretch-goal `Bot`, `Projectile`
- `Weapon` composed *into* `Player` (base class + `Rifle`, stretch `Grenade`)
- `NetworkManager` — owns the ENet host/peer lifetime via RAII, serializes/deserializes state packets
- `GameWorld` — owns entities in `std::vector<std::unique_ptr<Entity>>`, runs the update loop and collision checks
- `GameState` — Lobby / InGame / MatchOver state machine
- `Renderer` — thin SFML wrapper that draws from `GameWorld`

Data flow: input → local `Player` updates velocity/aim → `NetworkManager` serializes state → sent via ENet → remote `GameWorld` deserializes into a "remote" `Player` → `Renderer` draws all entities each frame.

Design choice: We use **composition** for `Player`-has-a-`Weapon` (a player can swap weapons; baking a weapon into the player's type via inheritance would be rigid), but **inheritance** for the `Entity` hierarchy, since polymorphic dispatch over one container of entities is the natural fit for a single game-update loop.

4. Tools

- **Build:** CMake (team is split Windows/Mac, so CMake keeps the build portable); compiled with g++/clang/MSVC depending on platform
- **Standard:** C++20
- **Libraries:** SFML (rendering/window/input), ENet (networking) — both pulled in via CMake `FetchContent` so a fresh clone builds without manual installs
- **Hardware:** standard laptops on the same LAN/Wi-Fi for multiplayer testing; no special hardware

- **Setup risk:** no prior SFML experience on the team. Mitigation: weeks 1-2 budgeted for tutorials + a minimal rendering prototype before building real features.
- **Fallback:** if networking or graphics proves too hard under deadline pressure, fall back to local 2-player hotseat (no network sync) or a console/ASCII-rendered version, to guarantee *something* demoable.

5. Work plan

- **Aditya** — core engine/game loop + `NetworkManager` (ENet wrapper), state sync protocol
- **Koushik** — graphics/rendering + player movement/input/HUD (SFML)
- **Khushal** — combat systems (weapon classes, hit detection), stretch-goal grenade physics and AI bots

Everyone writes tests for their own subsystem and reviews teammates' pull requests, so all three can explain the full program.

Week	Milestone
9 (now)	Proposal finalized; repo + CMake skeleton; each member completes an SFML/ENet intro tutorial
10 (Check-in 1)	Local 2-player movement + rendering working; first unit tests; networking integration begins
11	Player movement synced across 2 machines; hit detection begins
12 (Check-in 2)	Full core loop playable end-to-end; memory diagnostics run; tests updated
13	Stretch goals (bots / grenade physics) as time allows
14 (Readiness check)	Fresh-clone build test by one teammate using only the README; demo rehearsal
15	Final polish, report, slides, submission

Merge strategy: feature branches (`feature/networking` , `feature/rendering` , `feature/combat`), pull requests reviewed by at least one teammate before merging to `main` ; no direct commits to `main` .

6. Test plan and sources

- **Normal:** two connected clients, one fires at the other at close range → health decreases by the expected amount on both clients.
- **Boundary/empty:** a lobby with 0 or 1 connected players must not start a match.
- **Invalid input:** a malformed/out-of-range network packet is rejected by `NetworkManager` , not crashed on or desynced.
- **Failure condition:** a client disconnects mid-match → the server removes that entity and continues or ends the match gracefully, without crashing.

Sources: Project idea and scope were brainstormed and feasibility-checked with GitHub Copilot (AI coding assistant). No existing code or external tutorials used yet; specific tutorials adopted during implementation will be logged in the AI-use log as they come up.